Greedy

Greedy

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.

A **greedy algorithm** always makes the choice that looks best at the moment.

Greedy

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.

A **greedy algorithm** always makes the choice that looks best at the moment.

That is, it makes a **locally optimal choice** in the hope that this choice will lead to a **globally optimal solution**.

Greedy

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.

A **greedy algorithm** always makes the choice that looks best at the moment.

That is, it makes a **locally optimal choice** in the hope that this choice will lead to a **globally optimal solution**.

Question: Does greedy algorithm always yield optimal solution?

Coin changing

Input: A set of coins indicating currency denominations, c_1, c_2, c_3, …, c_n, and an integer x.

Question: devise a method to pay amount to customer using fewest coins.

Coin changing

Input: A set of coins indicating currency denominations, c_1, c_2, c_3, …, c_n, and an integer x.

Question: devise a method to pay amount to customer using fewest coins.

Greedy Choice:

At each iteration, add coin of the largest value that does not take us past the amount to be paid.

Coin changing

CASHIERS-ALGORITHM $(x, c_1, c_2, \ldots, c_n)$

SORT n coin denominations so that $0 < c_1 < c_2 < \ldots < c_n$.

$S \leftarrow \emptyset$. ⟵ multiset of coins selected

Coin changing

CASHIERS-ALGORITHM $(x, c_1, c_2, \ldots, c_n)$

SORT n coin denominations so that $0 < c_1 < c_2 < \ldots < c_n$.

$S \leftarrow \emptyset$. ⟵ multiset of coins selected

$k \leftarrow$ largest coin denomination c_k such that $c_k \leq x$.

Coin changing

CASHIERS-ALGORITHM $(x, c_1, c_2, \ldots, c_n)$

SORT n coin denominations so that $0 < c_1 < c_2 < \ldots < c_n$.

$S \leftarrow \emptyset$. ← multiset of coins selected

$k \leftarrow$ largest coin denomination c_k such that $c_k \leq x$.

IF (no such k)

    RETURN *"no solution."*

ELSE

    $x \leftarrow x - c_k$.

    $S \leftarrow S \cup \{ k \}$.

Coin changing

CASHIERS-ALGORITHM $(x, c_1, c_2, \ldots, c_n)$

SORT n coin denominations so that $0 < c_1 < c_2 < \ldots < c_n$.

$S \leftarrow \emptyset$. ← multiset of coins selected

WHILE $(x > 0)$

  $k \leftarrow$ largest coin denomination c_k such that $c_k \leq x$.

  IF (no such k)

    RETURN *"no solution."*

  ELSE

    $x \leftarrow x - c_k$.

    $S \leftarrow S \cup \{ k \}$.

RETURN S.

Question

Is the cashier’s algorithm **optimal**?

a) Yes, greedy algorithms are always optimal.
b) Yes, for any set of coin denominations $c_1 < c_2 < \ldots < c_n$ provided $c_1 = 1$.
c) None of them.

Question

Is cashier’s algorithm **optimal** for any set of denominations?

Input:

1, 2, 4, 6, 9, 10

x=15

Question

Is cashier’s algorithm **feasible** for any set of denominations?

Input:

2, 4, 7, 8, 20, 30

x=25

Question

Is cashier’s algorithm **feasible** for any set of denominations?

Input:

2, 4, 7, 8, 20, 30

x=25

Is it feasible at all ?

Question

Is cashier’s algorithm **feasible** for any set of denominations?

Input:

2, 4, 7, 8, 20, 30

x=25

Is it feasible at all ?

Input:

2, 4, 8, 20, 30

x=25

Infeasible

Question

Is cashier’s algorithm **feasible** for any set of denominations?

Input:

2, 4, 7, 8, 20, 30

x=25

Is it feasible at all ?

Input:

2, 4, 8, 20, 30

x=25

Infeasible

Can you make it feasible for any input by adding a small constraint?

Interval Scheduling

Interval Scheduling

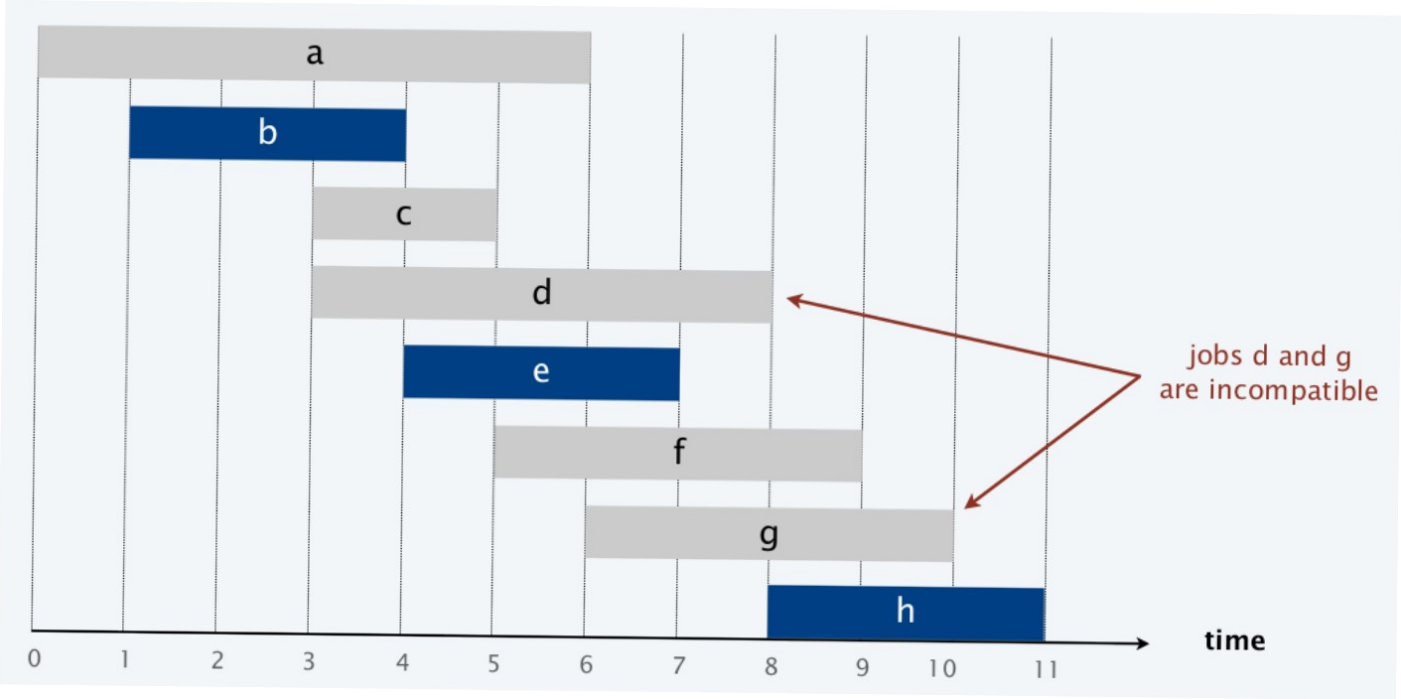
Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.

Interval Scheduling

Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.



Interval Scheduling

Question: Consider jobs in some order, taking each job provided it′s compatible with the ones already taken. Which rule is optimal?

A. [Earliest start time] Consider jobs in ascending order of s_j.

B. [Earliest finish time] Consider jobs in ascending order of f_j.

C. [Shortest interval] Consider jobs in ascending order of $f_j - s_j$.

D. None of the above.

Interval Scheduling

EARLIEST-FINISH-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT jobs by finish times and renumber so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

$S \leftarrow \emptyset$. ← set of jobs selected

Interval Scheduling

EARLIEST-FINISH-TIME-FIRST $(n, s_1, s_2, \ldots, s_n, f_1, f_2, \ldots, f_n)$

SORT jobs by finish times and renumber so that $f_1 \leq f_2 \leq \ldots \leq f_n$.

$S \leftarrow \emptyset$. ← set of jobs selected

FOR $j = 1$ TO n

  IF (job j is compatible with S)

    $S \leftarrow S \cup \{ j \}$.

RETURN S.

Interval Scheduling

Time: O(n log n) time.

1. Keep track of the job j^* that was added last to S
2. Job j is compatible with S if and only if $s_j \geq f_{j^*}$
3. Sorting by finish times takes O(n log n) time.

Interval Scheduling

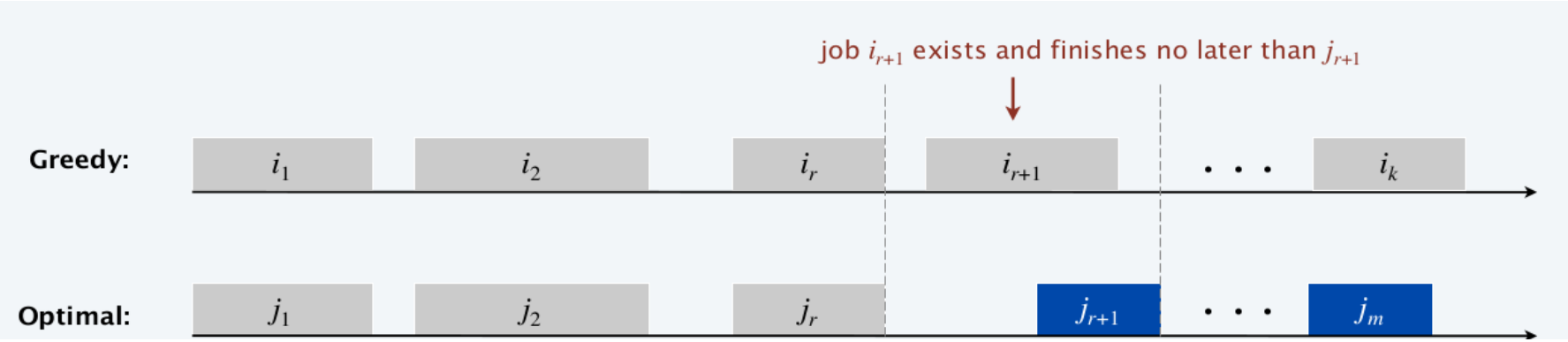
Correctness of Algorithm:

Suppose that greedy is not optimal. What happens?

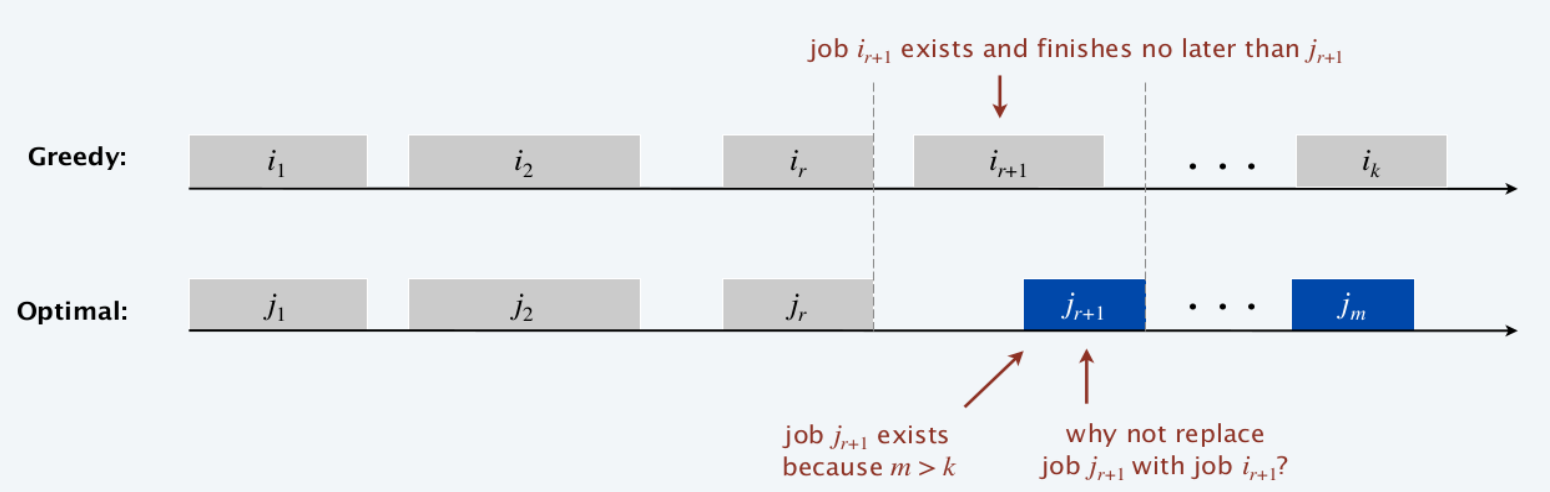
Interval Scheduling

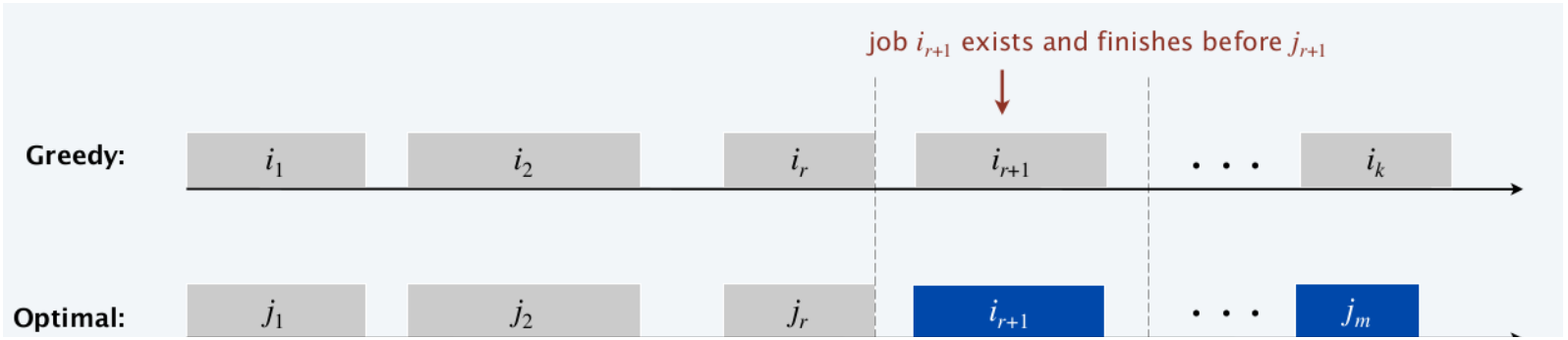
Correctness of Algorithm:

Suppose that greedy is not optimal. What happens?



Interval Scheduling





Interval Scheduling

Question: Suppose that each job also has a positive weight and the goal is to find a maximum weight subset of mutually compatible intervals. Is the **earliest-finish-time-first** algorithm still **optimal**?

Interval Scheduling

Question: Suppose that each job also has a positive weight and the goal is to find a maximum weight subset of mutually compatible intervals. Is the **earliest-finish-time-first** algorithm still **optimal**?

A. Yes, because greedy algorithms are always optimal.

B. Yes, because the same proof of correctness is valid.

C. No, because the same proof of correctness is no longer valid.

D. No, because you could assign a huge weight to a job that overlaps the job with the earliest finish time.

Greedy (3-steps) : general framework

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

Greedy (3-steps) : general framework

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

2) Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**. [Greedy Choice Property]

Greedy (3-steps) : general framework

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

2) Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**. [Greedy Choice Property]

3) Demonstrate **optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem. [Optimal Substructure Property]

Greedy (3-steps) : general framework

- $A_0 \xrightarrow{g_1} A_1 \xrightarrow{g_2} A_2 \xrightarrow{g_3} A_3 \xrightarrow{g_4} \ldots \xrightarrow{g_{p-1}} A_{p-1} \xrightarrow{g_p} A_p$

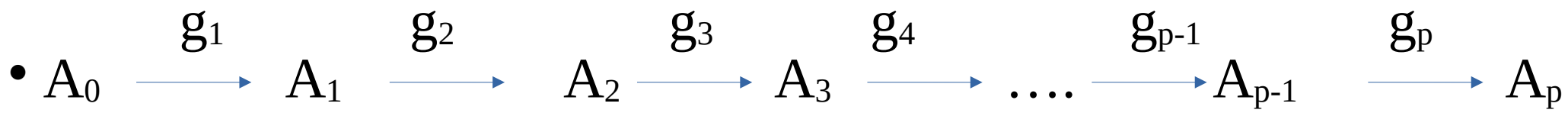
Greedy (3-steps) : general framework

- $A_0 \xrightarrow{g_1} A_1 \xrightarrow{g_2} A_2 \xrightarrow{g_3} A_3 \xrightarrow{g_4} \ldots \xrightarrow{g_{p-1}} A_{p-1} \xrightarrow{g_p} A_p$

What is the solution to the greedy algorithm?

When do you say that the algorithm is OPTIMAL?

Greedy (3-steps) : general framework



What is the solution to the greedy algorithm?

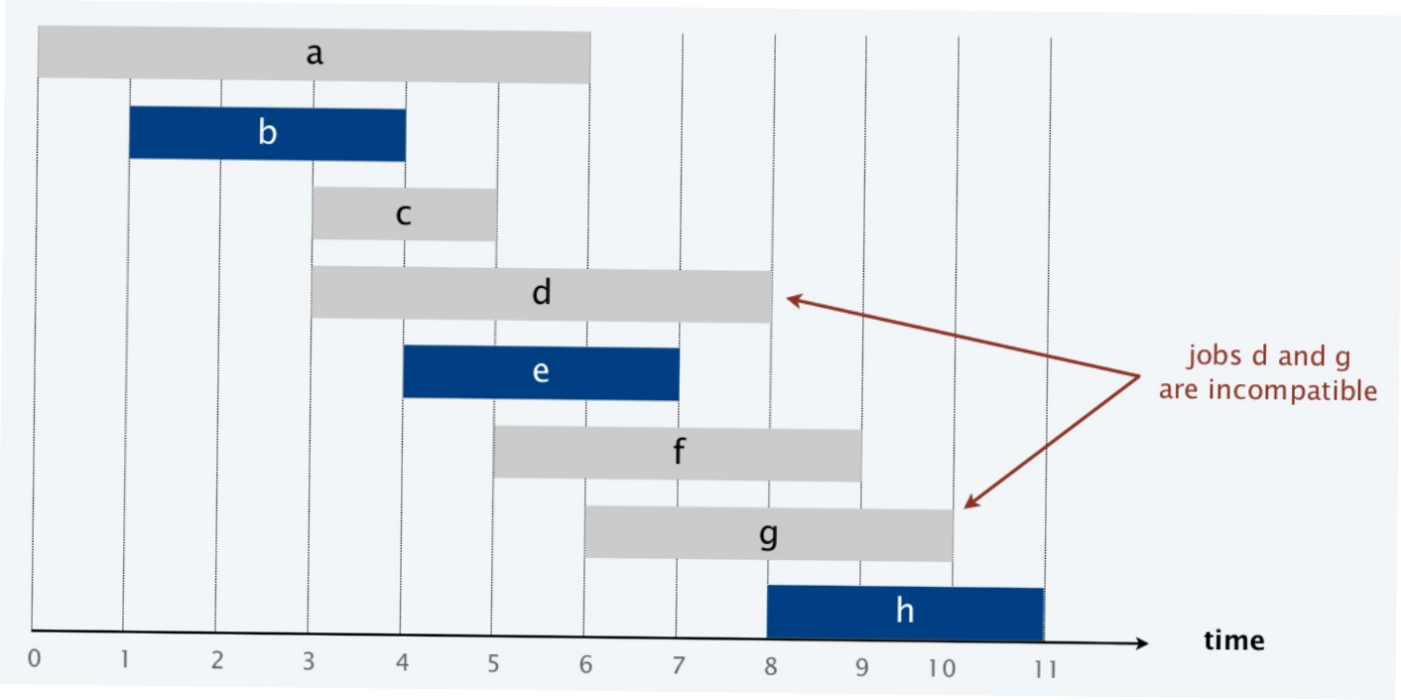
When do you say that the algorithm is OPTIMAL?

Demonstrate **optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that **if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.** [Optimal Substructure Property]

Interval Scheduling

Input: A set of jobs with start and finish times,

Question: find the maximum subset of mutually compatible (non-overlapping) jobs.



Another proof of Correctness Step 1.

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

”**Take the interval that finishes first and include in the solution**”.

What is the subproblem?

Another proof of Correctness Step 1.

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

”**Take the interval that finishes first and include in the solution**”.

Subproblem is after ignoring the closed neighborhood of the choosen interval.

Step 2: Greedy Choice Property.

Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**.

Step 2: Greedy Choice Property.

Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**.

”**There exists an optimal strategy that includes the interval that finishes first**”.

Step 3: Optimal Substructure Property.

Demonstrate **optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem.

”???”

Proof of Correctness (OPT. Sub. Prop.)

- I_{j^*} → interval with the earliest finish time.

  Intervals

- Optimal Solution

Proof of Correctness (OPT. Sub. Prop.)

- $I_{j^*} \rightarrow$ interval with the earliest finish time.

Intervals A

- Optimal Solution

Proof of Correctness (OPT. Sub. Prop.)

- I_{j^*} → interval with the earliest finish time.

Intervals A A'= A – X

where X is the set of intervals I_j such that $s_j < f_{j^*}$

- Optimal Solution

Proof of Lemma 2

- I_{j^*} → interval with the earliest finish time.

Intervals A A'= A – X

where X is the set of intervals I_j such that $s_j < f_{j^*}$

- Optimal Solution S'

Proof of Lemma 2

- I_{j^*} → interval with the earliest finish time.

Intervals A A'= A – X

where X is the set of intervals I_j such that $s_j < f_{j^*}$

- Optimal Solution S ← S'

Proof of Lemma 2

- I_{j^*} → interval with the earliest finish time.

Intervals A A'= A – X

where X is the set of intervals I_j such that $s_j < f_{j^*}$

- Optimal Solution S ← S'

$$S = S' + I_{j^*}$$

Proof of Lemma 2

- Suppose for a contradiction, let S is not an optimal solution.
- Let S* be an optimal solution. That is **|S*| > |S|** .

[Greedy is safe] There exists a solution S* that contains l_{j^*}

Proof of Lemma 2

- Suppose for a contradiction, let S is not an optimal solution.
- Let S* be an optimal solution. That is **|S*| > |S|** .

[Greedy is safe] There exists a solution S* that contains I_{j^*}

Now the subproblem is A’ and we have that $|S^* - I_{j^*}| > |S - I_{j^*}| = |S'|$

a contradiction to the fact that S’ was optimal in A’.